

# Buffer allocation as a linear program

RailCast — feasibility, formulation, and what the data supports

8 September 2026

**Purpose.** Assess whether an existing Irish Rail delay-prediction dataset supports a genuine mathematical-programming component: deciding how to distribute a fixed total schedule buffer across a train’s segments so that expected end-to-end lateness is minimised. The requirement is a formulation derived from scratch — decision variables, objective, constraints — and solved with a real solver, not a library call that happens to optimise internally.

## 1. What data we actually have

Everything needed is already in the archive. **Nothing new needs collecting.**

Per stop, `getTrainMovementsXML` gives:

| Field                                | Populated         |
|--------------------------------------|-------------------|
| ScheduledArrival, ScheduledDeparture | 95.4%             |
| Arrival                              | 69.1%             |
| Departure                            | 66.4%             |
| AutoArrival, AutoDepart              | with every actual |
| LocationOrder, LocationType          | $\approx 100\%$   |
| TrainOrigin, TrainDestination        | $\approx 100\%$   |

**Departure times are the unlock.** They were unused until now — the delay model only ever touched arrivals. Having both arrival and departure lets a journey be decomposed into *running time* (departure at stop  $i - 1$  to arrival at stop  $i$ ) and *dwell time* (arrival at stop  $i$  to departure at stop  $i$ ), which is exactly the decomposition buffer allocation requires.

Measured on the train and validation window, with the journey-consistency filter applied (583 of 16,776 journeys rejected):

- **211 segments** with  $\geq 30$  clean observations.
- **240 trains** with  $\geq 20$  days in which *every* segment is observed — a complete scenario set.
- **Slack** (scheduled minus 5th-percentile observed running time): median **90 s**, upper quartile 150 s, maximum 642 s. 146 of 211 segments exceed one minute.
- **Three segments have negative slack.** CRLow→ATHY is scheduled at 600 s; the 5th percentile of actual runs is 702 s. The timetable there is not achievable.

That last point matters: the problem is not purely academic. There is real, measurable, unevenly distributed slack.

## A caution about estimating the technical minimum

A first pass used the *minimum* observed running time as the technical minimum and produced ETOWN→MLGAR scheduled at 1050 s and apparently run in 156 s — roughly 580 km/h. ETOWN is one of four stations already identified as carrying arrival records belonging to other trains. Minimum-of-sample is not robust to that contamination; a 5th percentile is. Any use of a raw minimum in this work will reintroduce the same error.

## What is not available from the four endpoints in use

- connections between services, or which passengers transfer where;
- passenger counts or loadings;
- true minimum technical running times (these must be estimated);
- headways, track occupancy, platform capacity;
- rolling stock or crew circulation.

## 2. Which formulation the data supports

### Delay management: rejected

The delay-management problem — deciding which connections to hold and which to drop to minimise total passenger delay — requires two things the dataset does not hold: a definition of which services connect, and passenger weights on each connection.

Connections could be defined heuristically (two trains at one station within  $X$  minutes), but the passenger weights would be invented, and they sit *inside the objective function*. An optimum computed against fabricated weights is not defensible. Rejected on those grounds.

### Buffer allocation: supported

The data supports it well, and it is a genuine linear program.

One caveat to state plainly: this is a **counterfactual timetabling** exercise. Trains cannot be run on a re-buffered timetable, so evaluation is necessarily *simulation* — replaying historical primary delays through a propagation recursion under both the existing buffer vector and the optimised one. That is a standard and defensible method, but the recursion *is* a model, and its assumptions are the honest limitations of the result rather than a footnote.

### A correction on scenario generation

It is tempting to describe this as sample average approximation over the project's existing quantile predictions. That does not hold. The model predicts *total accumulated arrival delay at a stop given a vantage stop*, not per-segment primary delay. Differencing consecutive predictions does not recover a coherent joint scenario along a route, because the three quantile models were never fitted to be jointly consistent across stops.

The defensible scenario source is **historical per-segment excess running time**: whole days, so that within-day correlation is preserved by construction. That is genuine sample average approximation.

## 3. The formulation

### In plain terms

A timetable contains padding — scheduled running times are longer than the fastest a train can actually complete the segment. That padding absorbs delay. It is currently distributed however the timetable happens to distribute it. The question is whether, given the *same total* padding, there is a better place to put it so that trains arrive less late on average.

The decision is how many seconds of padding to give each segment. No padding can be invented; it can only be moved. Real days are then replayed through the result and the lateness counted.

## Index and data

Stops  $i = 0, 1, \dots, n$  along one train's route. Segment  $i$  runs from stop  $i - 1$  to stop  $i$ .

$$\begin{aligned} m_i &= \text{minimum technical running time on segment } i \\ r_i &= \text{scheduled running time on segment } i \\ b_i^0 &= r_i - m_i \\ B &= \sum_{i=1}^n b_i^0 \\ \delta_i^\omega &= \max(0, \rho_i^\omega - m_i) \end{aligned}$$

where  $\rho_i^\omega$  is the observed running time on segment  $i$  on day  $\omega$ .  $m_i$  is estimated as the 5th percentile of observed running times, for the robustness reason given in section 1.  $b_i^0$  is the timetable's existing buffer, which is also the baseline to beat.  $B$  is the budget: the same total buffer is redistributed, never added to, so end-to-end journey time is unchanged.  $\delta_i^\omega$  is the primary delay — the excess running time actually incurred on that segment that day.

## Decision variables

$$b_i \geq 0 \quad (i = 1 \dots n), \quad L_i^\omega \geq 0 \quad (i = 1 \dots n, \omega \in \Omega)$$

$b_i$  is the buffer allocated to segment  $i$ .  $L_i^\omega$  is arrival lateness at stop  $i$  under scenario  $\omega$ .

## Objective

$$\min_{b, L} \quad \frac{1}{|\Omega|} \sum_{\omega \in \Omega} \sum_{i=1}^n w_i L_i^\omega$$

## Constraints

$$\begin{aligned} L_i^\omega &\geq L_{i-1}^\omega + \delta_i^\omega - b_i && \forall i, \forall \omega \\ L_i^\omega &\geq 0 && \forall i, \forall \omega \\ \sum_{i=1}^n b_i &\leq B \\ 0 &\leq b_i \leq \bar{b}_i && \forall i \\ L_0^\omega &= \ell_0^\omega && (\text{initial delay; } 0 \text{ if the train starts on time}) \end{aligned}$$

## The linearisation, which is the part that matters

The true dynamics are

$$L_i^\omega = \max(0, L_{i-1}^\omega + \delta_i^\omega - b_i),$$

that is: a train carries its delay forward, buffer eats into it, and it cannot arrive “negatively late” in a way that would let it depart early. That max is piecewise-linear, not linear, and cannot enter a linear program as written.

The standard move is to replace the *equality* with the two *inequalities* above and let the objective do the rest. The argument that this is exact:

1. Each  $L_i^\omega$  enters the objective with coefficient  $w_i/|\Omega| > 0$ , so the solver drives it as small as possible.

2. Its only lower bounds are the two constraints. Pushed down, it lands exactly on the value  $\max(0, L_{i-1}^\omega + \delta_i^\omega - b_i)$  that the true recursion specifies.
3.  $L_i^\omega$  also appears on the right-hand side of the constraint for  $L_{i+1}^\omega$ , with a *positive* coefficient — so reducing it *relaxes* the next constraint. Both forces act in the same direction, giving no incentive to inflate it.

The linear program’s optimum therefore reproduces the nonlinear recursion exactly.

**The failure mode to know.** This holds only while every  $w_i \geq 0$ . Introduce a negative weight anywhere — for instance to “reward early arrival” — and the relaxation breaks: the solver inflates that  $L_i^\omega$  to buy an improvement elsewhere, and the lateness variables stop representing lateness. Rewarding earliness requires a different encoding, not a negative weight.

## Choosing the weights

$w_i$  is a modelling decision. Setting  $w_i = 0$  for all stops but the terminus optimises end-to-end punctuality only;  $w_i = 1$  everywhere optimises for passengers alighting anywhere along the route. Weighting by boardings would be preferable, and boardings are not available.

## Size and solver

For a 29-stop DART service with 32 complete days: 28 buffer variables,  $28 \times 32 = 896$  lateness variables, and roughly 1,820 constraints. This solves in milliseconds.

`scipy.optimize.linprog` with the HiGHS backend is already installed as a transitive dependency, so no new package is required. The constraint matrix should be built sparse; its structure is very sparse and a dense representation wastes memory for no benefit at larger scale.

## 4. Baseline and limitations

**Baseline.**  $b^0$ , the timetable’s own buffer vector, evaluated with the same recursion on **held-out days**. Buffers are fitted on one set of days and evaluated on another. Optimising and evaluating on the same days measures only the ability to fit noise.

**Limitations, to be published alongside whatever number results:**

1. **Primary delays are assumed exogenous** — that  $\delta_i^\omega$  does not change when  $b_i$  changes. In practice drivers pace to the timetable, so a segment given more buffer may simply attract a slower run. This is the largest threat to the result and cannot be tested with this data.
2. **Minimum running time is estimated**, not known. The result moves with the choice of percentile; the sensitivity should be reported.
3. **Single train, no network.** A re-buffered service could conflict with another on shared track. There are no headway or platform constraints.
4. **Counterfactual.** The result cannot be validated against real operations, only simulated.
5. **Dwell buffer is ignored** in the base formulation; buffer lives only on running segments.

**One assumption that is *not* required:** the program does not assume independence between segments. Because each scenario is a whole day, within-day correlation is preserved exactly. This is a genuine advantage of sample average approximation over a parametric approach.

---

*If the optimiser does not beat the existing timetable, that result is recorded and kept. A negative result honestly measured is worth more than a claim that has to be hedged.*